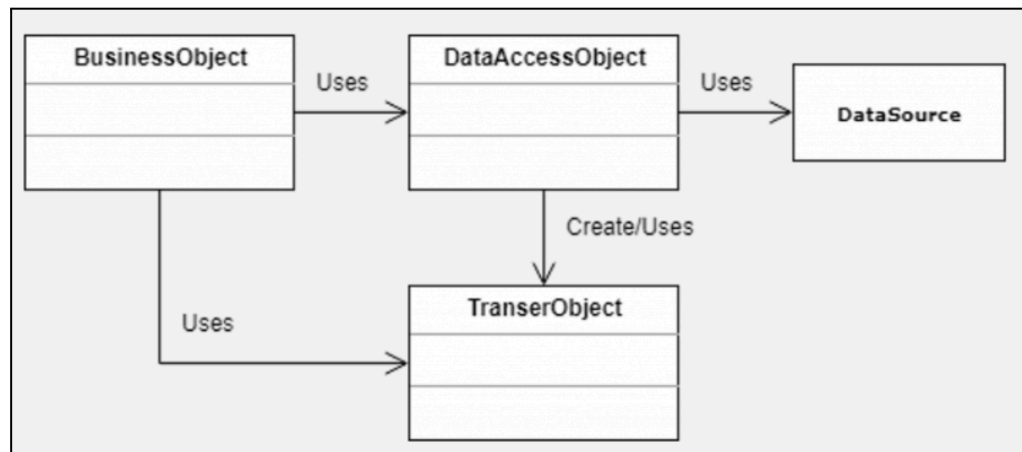


b) Investigación:

Patrón DAO: ➤

- ¿Qué es un Patrón en desarrollo de software?
Es una solución general y reutilizable para un problema que aparece frecuentemente al desarrollar sistemas. Una forma de organizar las clases, objetos y responsabilidades para resolver un problema de manera estructurada.
- ¿Qué es el patrón de acceso a datos DAO? ¿Para qué sirve? Realizar de forma gráfica su funcionamiento.

DAO significa Data Access Object (Objeto de Acceso a Datos). Es un patrón que se utiliza para separar la lógica de acceso a datos del resto de la aplicación. La idea principal es: "El código que necesita datos no debería preocuparse por cómo se obtienen esos datos."



Object Relational Mapping (ORM):

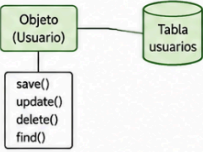
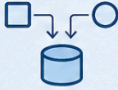
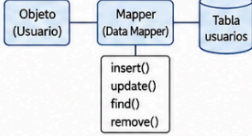
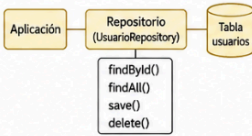
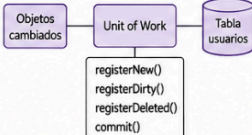
- ¿Qué es un ORM?
 - o ORM son las siglas de Mapeo de Objetos Relacionales, tecnica que permite trabajar con una DB relacional usando objetos y clases del lenguaje de programación, en lugar de escribir SQL constantemente.
- ¿Cuáles son los ORM más conocidos del mercado?
 - o Hibernate (JAVA) : ORM mas utilizado en Java, implementa JPA
 - o SQLAlchemy (Python)
 - o Sequelize ()
 - o Entity Framework Core (C# / .NET) : ORM oficial y muy usado en el ecosistema .NET.
- ¿Qué lenguajes de programación pueden interactuar con ORM?
 - o Python, Java, C# son lenguajes de programación que pueden interactuar con los ORMA mas usados en la actualidad.
- ¿Cómo funciona un ORM?

- Se trata de un intermediario entre la DB Relacional y una aplicación, permite la manipulación de registros (CRUD) mediante objetos, ocultando la capa SQL y capas de seguridad de la DB
- ¿Cuáles son sus ventajas y desventajas?
 - Ventajas:
 - § No necesidad de usar Lenguaje SQL
 - § Trabaja con Objetos, facilita el aprendizaje
 - § Facilita el entendimiento
 - Desventajas:
 - § Una consulta en ORM puede internamente generar SQL ineficiente
 - § Curva de aprendizaje
 - § Rendimiento
 - § Dificultad en operaciones complejas
- Realizar de forma gráfica su funcionamiento



Patrones ORM:

Cuadro comparativo:

Patrones principales utilizados en ORM			
PATRÓN	¿QUÉ HACE?	¿CÓMO SE APLICA EN UN ORM?	EJEMPLO EN UN ORM
 ACTIVE RECORD	El objeto representa un registro de la base de datos y contiene métodos para guardarlo, modificarlo o eliminarlo.	Cada objeto incluye la lógica necesaria para persistir sus propios datos en la base de datos. 	El objeto se encarga de persistir sus datos. <pre> usuario = Usuario() usuario.nombre = "Juan" usuario.save() </pre>
 DATA MAPPER	Separa los objetos del programa de la base de datos. Un componente externo realiza el mapeo entre objetos y tablas.	Un mapper (o componente de mapeo) se encarga de convertir objetos en registros de la base de datos y viceversa. 	El mapper realiza las operaciones de persistencia. <pre> mapper = UsuarioMapper() mapper.save(usuario) usuario = mapper.find(5) </pre>
 REPOSITORY	Proporciona una interfaz para consultar y guardar objetos sin que el programador tenga que utilizar SQL.	El repositorio ofrece métodos para acceder a la colección de objetos del dominio, ocultando los detalles de la fuente de datos. 	El programador trabaja con métodos del repositorio. <pre> repo = UsuarioRepository() usuario = repo.findById(5) repo.save(usuario) </pre>
 UNIT OF WORK	Registra los cambios realizados en los objetos y los guarda en la base de datos como una única operación.	Administra los objetos nuevos, modificados o eliminados y sincroniza todos los cambios con la base de datos al confirmar la transacción. 	Se guardan todos los cambios al confirmar la unidad de trabajo. <pre> uow.registerDirty(usuario) uow.registerNew(otroUsuario) uow.commit() </pre>

Java Persistence Api (JPA):

¿Qué es JPA?

JPA (Java Persistence API) es una especificación de Java para trabajar con bases de datos relacionales mediante objetos.

Su objetivo es permitir que podamos representar las tablas de una base de datos como clases Java, y los registros como objetos, sin tener que escribir todo el SQL manualmente.

¿Cuáles son las principales características de JPA?

1. Mapeo objeto-relacional (ORM): Permite relacionar
 - Java | Base de datos
 - Clase → Tabla
 - Objeto → Registro
 - Atributo → Columna
2. Permite trabajar con objetos: podemos trabajar con objetos Java y dejar que el framework genere las consultas necesarias.
3. Persistencia: Permite guardar objetos Java en una base de datos y posteriormente recuperarlos.
4. Manejo de relaciones: @OneToMany, @ManyToOne, @OneToOne, @ManyToMany.

¿Qué significa que una clase sea de tipo Entidad?

Una Entidad (Entity) es una clase Java que representa un objeto que queremos persistir en la base de datos. Cada objeto representa potencialmente un registro de esa tabla.

¿Cuáles son las anotaciones que se pueden utilizar en una clase de tipo entidad?

Las más comunes son:

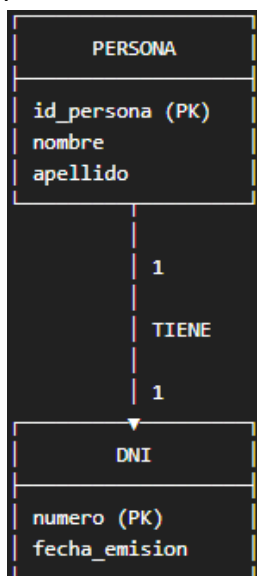
Anotación	Función
<code>@Entity</code>	Indica que la clase es una entidad JPA
<code>@Table</code>	Permite especificar la tabla de la BD
<code>@Id</code>	Indica el atributo que funciona como clave primaria
<code>@GeneratedValue</code>	Indica que el valor del ID se genera automáticamente
<code>@Column</code>	Permite configurar una columna
<code>@Transient</code>	Indica que un atributo no debe persistirse
<code>@OneToOne</code>	Relación uno a uno
<code>@OneToMany</code>	Relación uno a muchos
<code>@ManyToOne</code>	Relación muchos a uno
<code>@ManyToMany</code>	Relación muchos a muchos
<code>@JoinColumn</code>	Configura la columna utilizada para una relación

¿Cuáles son las anotaciones que se pueden utilizar para validar información en un atributo de JPA? Las más comunes son:

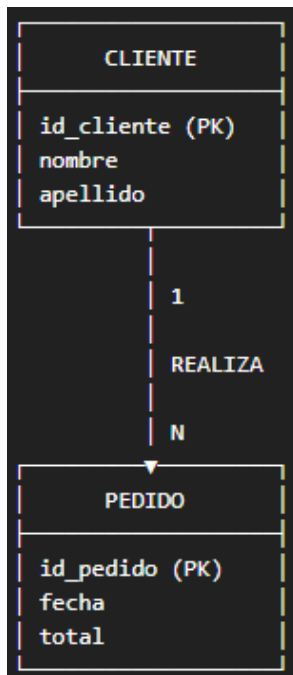
Anotación	¿Qué valida?
<code>@NotNull</code>	No permite <code>null</code>
<code>@NotBlank</code>	No permite <code>null</code> , vacío ni espacios
<code>@NotEmpty</code>	No permite <code>null</code> ni vacío
<code>@Size</code>	Tamaño mínimo/máximo
<code>@Min</code>	Valor mínimo
<code>@Max</code>	Valor máximo
<code>@Positive</code>	Debe ser positivo
<code>@PositiveOrZero</code>	Debe ser positivo o cero
<code>@Negative</code>	Debe ser negativo
<code>@Email</code>	Debe tener formato de email
<code>@Pattern</code>	Debe cumplir una expresión regular
<code>@Past</code>	Fecha en el pasado
<code>@Future</code>	Fecha en el futuro
<code>@DecimalMin</code>	Valor decimal mínimo
<code>@DecimalMax</code>	Valor decimal máximo

➤ ¿Cuáles son las relaciones entre clase que se pueden anotar en JPA para ser mapeada a una base de datos? Dar un ejemplo sencillo de cada una de las relaciones y realizar el DER para cada ejemplo

@OneToOne: Significa que una instancia de una clase se relaciona con una sola instancia de otra clase. Por ejemplo: Una Persona tiene un único DNI, y ese DNI pertenece a una única Persona.



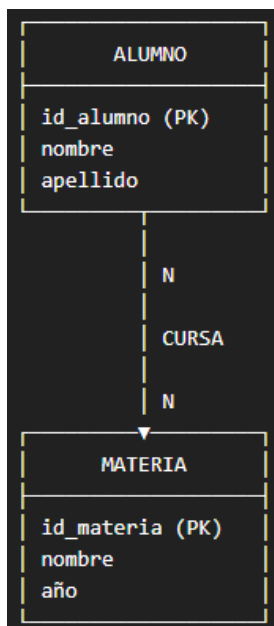
@OneToMany — Cliente y Pedido: Significa que una instancia de una clase puede estar relacionada con muchas instancias de otra clase. Por ejemplo: Cliente – Pedido



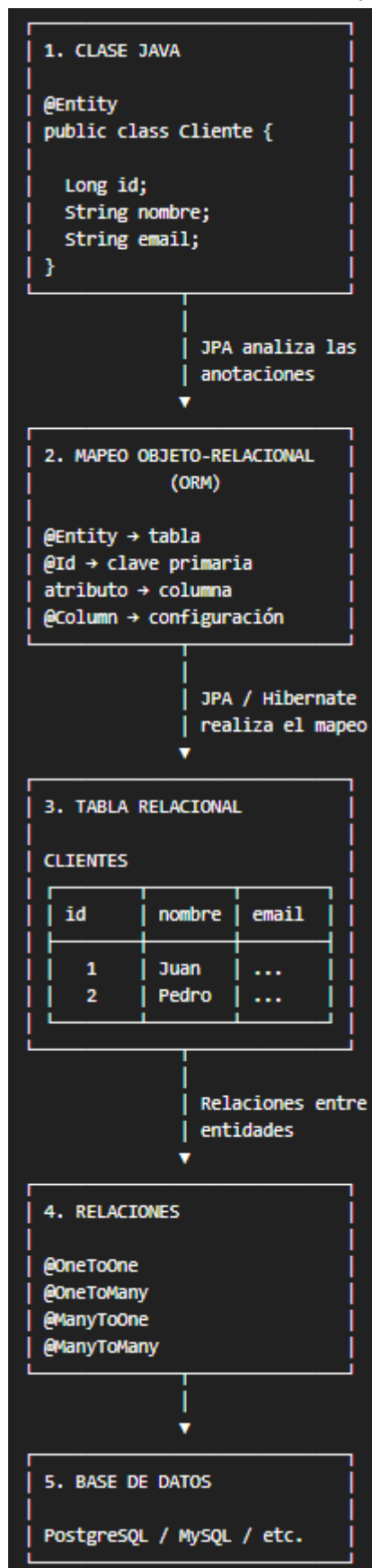
@ManyToOne — Pedido y Cliente: Aquí hay que aclarar algo importante:

@OneToMany y @ManyToOne pueden representar la misma relación desde perspectivas diferentes. Por lo que podemos tomar el ejemplo anterior

@ManyToMany — Significa que muchas instancias de una clase pueden relacionarse con muchas instancias de otra clase. Por ejemplo: Alumno y Materia, un alumno puede cursar muchas materias.



➤ ¿Graficar paso por paso cómo es el funcionamiento de JPA para el Mapeo de una clase Entidad hasta su conversión en tabla y sus respectivas relaciones?



➤ ¿Qué es un JPQL? ¿Qué similitudes y qué diferencias existen con una sentencia SQL?.
Dar 5 ejemplos de JPQL y su respectiva sentencia SQL.

- JPQL es un lenguaje de consultas definido por JPA que permite realizar consultas sobre las entidades y atributos de Java, en lugar de consultar directamente las tablas y columnas de la base de datos.
- JPQL y SQL se parecen bastante porque ambos permiten realizar consultas sobre datos. Ambos utilizan: SELECT, FROM, WHERE, ORDER BY, GROUP BY, permiten consultas con SUM, AVG, COUNT, realizar consultas sobre varias entidades (tablas), utilizar condiciones con el WHERE, entre otras.
- La diferencia más importante es qué estamos consultando. SQL trabaja directamente con tablas y columnas, JPQL trabaja con entidades y atributos. Otra diferencia importante es que JPQL es independiente de la base de datos, cualquier consulta será traducida por JPA/Hibernate en SQL para Postgre o MySQL o el SQL correspondiente.

Ejemplo 1 — Obtener todos los clientes

- JPQL: `SELECT c FROM Cliente c`
- SQL: `SELECT * FROM cliente;`

Ejemplo 2 — Buscar clientes mayores de 18 años

- JPQL: `SELECT c FROM Cliente c WHERE c.edad > 18`
- SQL: `SELECT * FROM cliente WHERE edad>18`

Ejemplo 3 — Buscar un cliente por nombre

- JPQL: `SELECT c FROM Cliente c WHERE c.nombre = 'Juan'`
- SQL: `SELECT * FROM cliente WHERE nombre = 'Juan';`

Ejemplo 4 — Ordenar clientes por nombre

- JPQL: `SELECT c FROM Cliente c ORDER BY c.nombre ASC`
- SQL: `SELECT * FROM cliente ORDER BY nombre ASC;`

Ejemplo 5 — Contar clientes

- JPQL: `SELECT COUNT(c) FROM Cliente c`
- SQL: `SELECT COUNT(*) FROM cliente;`

d4) ¿Cuál es la configuración en JPA que se puede utilizar para conocer cómo el motor de persistencia transforma los JPQL en sentencia SQL?

La configuración que se utiliza para ver cómo JPA/Hibernate transforma las consultas JPQL en sentencias SQL es la propiedad: **spring.jpa.show-sql=true**

Dar un ejemplo del JPQL para la obtención de Factura y como queda esa sentencia expresada en SQL.

- Ejemplo: Supongamos que tenemos una entidad Factura,

```
@Entity
public class Factura {

    @Id
    private Long id;

    private String numero;
    private double total;
}
```

Y queremos obtener todas las facturas:

```
SELECT f FROM Factura f
```

Esta consulta trabaja con la entidad Factura, no directamente con la tabla. Hibernate podría transformarla en un SQL similar a:

```
SELECT f1_0.id, f1_0.numero, f1_0.total FROM factura f1_0;
```